Umpyr Resolve — Business Logic and How It Works

As built on 2026-10-20. Companion: "Umpyr Resolve — Technical Architecture".

1. What Umpyr Resolve does

A customer writes to a merchant about an order or a booking: a damaged item, a late flight, a return, a refund. Umpyr reads the message, finds the merchant's record, works out what the merchant's written policy and the law provide, checks that against how the merchant's own staff have handled similar cases, and answers. Where the answer is clear and within limits, it resolves. Where only the customer can decide, it lays out the options. Where a person must sign off, it queues the case with everything the reviewer needs. Where neither policy nor precedent covers the situation, it hands the case to a person with the evidence attached. Everything it did, and why, is written to a tamper-evident log, and every policy statement it makes points to the document, section and page it came from.

2. The rules Umpyr lives by

1. **Policy decides.** The outcome of a case comes from the merchant's compiled policy and the regulation above it. A language model never decides a remedy or an amount; it only reads, structures and writes.
2. **Regulation outranks the merchant.** Rules carry a tier: law, then contract (terms, contract of carriage), then the merchant's published commitments, then goodwill. A US DOT refund right beats a stricter airline term.
3. **Never assume.** A fact is either on the merchant's record or in the customer's own words. If the policy turns on a choice only the customer can make, Umpyr presents the choice; it does not pick for them.
4. **Never invent.** Umpyr never names a product, flight, price or reason the customer did not give. A cloned record for an unknown reference is scrubbed of everything that was not stated.
5. **Never loop.** A fact that has been asked twice and still is not captured goes to a person. The same question is never asked a third time.
6. **Cite, don't generate.** Every policy-based statement shown to a reviewer carries the document version, section, page and clause. Customers see public documents only, never internal procedures.
7. **Precedent verifies; it does not override.** How staff handled similar cases shapes the plan and flags disagreements, and it is what a person sees when policy is silent. It cannot by itself overrule written policy.
8. **Record everything.** One case per conversation, one hash-chained event per step, one commitment hash per decision.

3. Life of a case

Each customer message is one turn. On every turn Umpyr:

1. **Masks** personal data and opens or reloads the case.

2. **Reads** the message into structured facts: what the customer wants (intents), the reference, how many people, the claim and its reason, flight or bag facts, whether they asked about timing. From the second message on it reads only what changed.
3. **Finds the record** by order number or confirmation code. The merchant's record is itself proof of purchase.
4. **Fills in what the evidence implies**: a bag complaint is about a checked bag; a missing wheel is damage; "unopened in the box" means unused; speaking to the airport agent counts as reporting the bag. A delay cause is only kept when the customer named it.
5. **Plans** two ways and compares: what the merchant's staff did in the five most similar past cases, and what a model proposes from a fixed list of actions. Agreement is recorded; a real disagreement on the remedy is a flag.
6. **Applies the rules.** The result is one of: resolved, denied, needs a fact, needs the customer's choice, pending approval, escalated.
7. **Checks authority.** Is the action licensed by the rule, within the ruled amount, within the merchant's auto-approve limit and the session's mode? Would an irreversible money action go against confident precedent? If so, a person confirms first.
8. **Handles follow-ups**: a new ask is classified on its own; an ask policy does not cover is checked against precedent and, if staff have granted goodwill in similar denied cases, sent to a person with that evidence; a second pushback on a denial goes to a person regardless.
9. **Answers** with a drafted reply that a second model checks against the ruling and the customer's message. If the check fails twice, a plain template is used.
10. **Writes the record**: the per-turn ruling with its hash, the ledger events, the decision commitment.

Three real conversations

Damaged stroller, Alaska. "One wheel is missing, the agent at the airport says they cannot refund, confirmation XVXXT, what are my options?" The record is unknown, so a stand-in booking is created with a checked, damaged bag. The Contract of Carriage, Rule 15, excludes damage to protruding parts such as wheels; the rules deny on the first turn, citing Rule 15 with page numbers. When the customer pushes back and asks for credit, Umpyr finds that staff granted a voucher in 2 of 22 similar denied cases and hands the case to a specialist with those two tickets attached, telling the customer a specialist is reviewing whether goodwill applies. Before these rules, this conversation looped six times asking for facts the customer had already given.

Delayed flight, Alaska. "Kept delaying, now 8 PM, no ETA, what are my options?" Only the customer can decide whether to travel. Umpyr presents the two branches the policy provides: a full refund if they do not travel, marked as their right under the DOT rule, or rebooking on the next flight. Nothing is decided or queued. When the customer asks for a morning flight, a hotel and meals, the rebooking is granted under the airline's delay commitment and the hotel and meals are named as a separate matter for a specialist, because the cause of the delay was never stated and the expense rules turn on it.

Returned desk, Rockler. "I lost the receipt but have the card bill, what are my options?" The matched order is proof of purchase, so no denial for a missing receipt. Umpyr asks the two things the return policy turns on: unopened, used. Once answered, the policy provides a refund to the original card; because Rockler's own staff usually deny change-of-mind returns, the refund is queued for a person with a 24-hour promise to the customer. "Good, I like you" gets a one-line thank-you and nothing is re-run.

4. Policy: from documents to rules

- Policy sources are fetched and archived with a content hash. A model splits them into verbatim clauses and proposes a rule for each: the scope (which dispute types), the facts it needs, the condition, and the effect (decision, remedy, amount formula). A reviewer approves, patches or adds rules; the approved set is the rule pack, versioned by date.
- Each clause keeps its provenance: document, version, section heading, PDF page, exact character span, and a hash of its text. Citations anywhere in Umpyr are built from this, never written by a model.
- A regulation layer sits above merchant policy. When an undecided regulation could change the outcome and a fact is missing, Umpyr asks for the fact before letting a lower-tier rule decide.
- Where the compiler reads a clause too narrowly, the merchant adds the missing reading as an override. The Alaska "choices during a delay" commitment had been compiled as a refund rule only; the rebooking branch was added that way.

5. Precedent: how history is used

- Historical tickets are mined into structured cases: what was asked, what facts applied, what staff did step by step, what the outcome was, and whether that outcome matched compiled policy (drift).
- At runtime the five most similar cases give a consensus plan and a confidence. History is confident when enough neighbours agree on the remedy.
- Precedent acts in three places: it shapes the plan; a confident precedent that contradicts the ruling's remedy holds an auto-approved money action for a person; and when policy denies and the customer asks for more, precedent of staff goodwill in similar denied cases is what the reviewer sees.
- Practice that staff follow consistently but no clause states becomes an "unwritten rule" candidate the merchant can adopt as written policy.

6. Outcomes and what the customer sees

Status	Meaning	Customer sees
resolved	Policy provides the remedy and it is within authority	The remedy, the amount, the public source, a cited timeline if they asked when the money lands
denied	Policy excludes it	A plain explanation with the public source, and an invitation to correct any misunderstanding
needs_fact	A fact the rules need is missing	One short question per fact, with what it decides
needs_choice	Only the customer can decide	The options, the regulated one marked as their right, nothing queued
pending_approval	A person must authorise	"Sent for authorisation", the promised window, and later an honest notice if it is late
escalated	A person must decide	"A specialist has your case"; no speculation about the outcome

Customer-facing conventions: a status strip of what was done ("Checked your booking · Reviewed what the policy provides · Sent for authorisation"), public sources under the reply, buttons for choices and refund options, no repetition of policy text already given, no re-evaluation of thank-you messages.

7. Governance and merchant controls

- **Modes**: autonomous, assist (every action queued), shadow (decide, never act).
- **Auto-approve limit** per session (default $100); above it, a person authorises.
- **Approval SLA** per workspace (Alaska 4 hours, Rockler 24): the customer is told the window; an overdue queue escalates with an honest message.
- **Planner gate**: confident precedent contradicting the ruling on a money action holds it for review.
- **Supervisor**: every reply is checked against the ruling and the customer's message; wrong or unsupported statements are rejected, omissions are not.
- **Typed escalations**: data gap (no record, missing fact, no precedent), policy gap (no rule, conflict, uncovered ask), planner disagreement, governance gate (limit, mode, overdue). The metrics endpoint reports rates by type.

8. Audit and evidence

Every conversation leaves: the masked messages; one ruling per turn with its canonical hash, the rule pack version, the facts, the checks and the authorisation; a hash-chained event log with actor, action, basis and result for every stage, including the clause hashes behind the decision; a decision commitment hash; and the cost of every model call attributed to the case. The chain is append-only at the database level and can be re-verified at any time. The same record serves the merchant's audit, a chargeback evidence pack, and the training data for the learning loop.

9. Learning loop: today and planned

Today the precedent store is the mined history, and live cases do not yet feed back. The planned loop (phases 4 and 5):

- Every resolved case, whether policy or a person decided it, becomes a precedent with its outcome.
- A person's decision on a queued or escalated case is a label: approve, reject, or a different remedy with a reason.
- Labels update the precedent store, the scorecards that let a workflow graduate to automatic execution, and the candidate rules a merchant can adopt as written policy.
- Downstream outcomes (reopens, chargebacks after resolution, actual settlement time) are the strongest signals.

This is a supervised feedback loop over labelled decisions, not reinforcement learning in the policy-gradient sense; for anything that moves money, that is the safer design.

10. Deliberately not done yet

Execution against merchant systems (refunds are decided and queued, not paid); the approval queue UI and decision cards; the settlement-time model; policy-document change detection; identity, ownership

and abuse checks (Umpyr Check, hooks reserved); streamed interim status; multi-remedy rulings.

11. Glossary

Case — one conversation and its state. **Clause** — a verbatim policy statement with provenance. **Rule** — an executable reading of a clause. **Rule pack** — the versioned, approved set of rules. **Tier** — law, contract, commitment, goodwill. **Ruling** — the engine's outcome for a turn. **PDP** — the authorisation check before any action. **Ledger** — the hash-chained event log per case. **Precedent** — mined historical cases and their outcomes. **Drift** — a historical outcome that contradicts compiled policy. **Stand-in record** — a POC record cloned for an unknown reference. **Choice fact** — a fact only the customer can supply, presented as options.